

Project Progress Report

Amity School of Engineering & Technology

Project Group ID:	265	Project Area:	Data Science / Machine Learning
Programme:	B. Tech (CSE) – VIII Semester	Session:	2025–2026
Report Type:	Project Progress Report		

Project Title: Machine Learning based Predictive Model for Livability using Environmental and Socioeconomic indicators

Students:

Name	Roll No.	Enrollment No.
Sagar Singh Bisht	056	A023167022056
Aakarsh Pab	050	A023167022050

1. Synopsis & Scope Refinement

The project scope was refined from a generic Liveability Predictor to a specialised Decision Support System — CityMatch — capable of solving multi-criteria urban relocation as an AI-driven optimisation problem.

- **Pivot:** Shifted from descriptive analytics (visualising data) to prescriptive analytics (recommending locations based on user priorities).
- **Objective:** Mathematically match user preference vectors (Budget, Health, Commute, Amenities) with neighbourhood attribute vectors derived from live and historical data sources.
- **Scope Expansion:** Coverage extended from an initial 50 neighbourhood centroids to **72 verified residential localities** across Delhi, with three peri-urban outliers (Najafgarh, Narela, Bawana) explicitly excluded after geospatial profiling confirmed they are not comparable to urban residential areas.
- **Scoring Correction:** Replaced the original Cosine Similarity engine with a **Weighted Sum scoring formula** after identifying that cosine similarity produced directionally incorrect rankings — neighbourhoods scoring highest on a single factor outranked balanced matches. The new engine applies per-factor directional inversion and weight normalisation, validated through three boundary-condition tests.

2. Literature Review & Methodology Selection

A targeted review of unsupervised learning techniques for urban segmentation informed all major algorithm and feature engineering decisions.

- **Algorithm Retained:** K-Means Clustering (K=4) was confirmed as optimal over Hierarchical Clustering after Elbow Method and Silhouette Analysis (S=0.41) identified a clear four-cluster solution representing distinct Delhi neighbourhood archetypes.
- **Scoring Engine Revision:** Cosine Similarity was replaced by a Weighted Sum formula following boundary-condition testing that exposed directional ranking errors. The new engine correctly handles inverse features (Rent, Pollution, Traffic) by applying $f(x) = 1 - x$ before scoring.
- **Rent Data Source Upgraded:** Manually estimated 2BHK rent ranges were replaced with median rents aggregated from 5,902 verified listings drawn from a public Kaggle dataset of Delhi property listings — substantially improving data quality and ML model validity.
- **Traffic Sampling Strategy Formalised:** A 16-reading sampling protocol was designed (4 days x 4 time slots: 09:00, 13:00, 18:00, 23:00 IST) to construct a statistically stable daily average congestion index, replacing the earlier peak/off-peak two-slot proxy.

3. Data Sourcing & API Integration

A four-source data pipeline was engineered to overcome the absence of any single unified urban dataset for Delhi at neighbourhood granularity:

TomTom Traffic Flow API (v4): Fetches current speed vs. free-flow speed for the road segment nearest each neighbourhood centroid. A Congestion Index $CI = (1 - v_current / v_freeflow) \times 10$ is computed per reading and averaged across 16 samples per neighbourhood (4 days x 4 time slots), producing a robust daily average traffic profile.

CPCB Air Quality Data: PM2.5 concentrations split into two seasonal averages — Winter (October–February) and Monsoon (June–September) — to capture the bimodal pollution cycle of the Indo-Gangetic Plain. Seasonal decomposition replaced the single annual PM2.5 average used in earlier iterations.

OSMnx (OpenStreetMap): Amenity density scores — schools, hospitals, metro stations, parks — extracted within a 1 km radius of each neighbourhood centroid via the Overpass API. Raw counts are subject to 95th-percentile capping before scaling to correct for OpenStreetMap tagging inconsistencies (e.g., health dispensaries tagged as hospitals).

Kaggle Rental Dataset: 17,890 Delhi property listings filtered to 2BHK configurations (n=5,902). Median monthly rent per locality computed and matched to 72 neighbourhoods via direct string matching (48 localities) and manually verified fuzzy matching (24 localities).

4. Data Preprocessing & Feature Engineering

Outlier Capping: All amenity count features (schools, hospitals, metro stations, parks) clipped at the 95th percentile prior to scaling. This correction was required after one peripheral neighbourhood (Najafgarh) recorded 17 hospitals — the dataset maximum — due to small dispensaries being tagged identically to full hospitals in OpenStreetMap. Without capping, this single outlier dominated recommendations regardless of user preferences.

Normalisation: Min-Max Scaling applied to all features after capping, normalising disparate units (Rent in INR, Traffic Index 0-10, PM2.5 in $\mu\text{g}/\text{m}^3$, amenity counts) into a uniform $[0, 1]$ vector space. Scaling is recomputed from scratch whenever the dataset is updated to prevent stale scaled columns.

Feature Inversion: Three features where lower values indicate better liveability — PM2.5 (both seasons), Traffic Congestion Index, and Rent Price — are transformed as $f(x) = 1 - x_scaled$ before scoring. This ensures the scoring direction is uniformly higher = better across all six dimensions.

Rent Column Reconstruction: The original Rent_Price column contained manually estimated midpoints of rent ranges (e.g., midpoint of 25,000-38,000). These were replaced with Kaggle-sourced median rents per locality, improving data representativeness and increasing the Random Forest rent prediction model's coverage.

Peri-Urban Exclusion: Three neighbourhoods — Najafgarh, Narela, Bawana — excluded from the final model as peri-urban agricultural settlements that are not meaningfully comparable to urban residential areas. This decision is explicitly documented as a data scope criterion.

5. Modelling & Implementation

K-Means Clustering Engine: K-Means (K=4) applied to six scaled liveability features: PM2.5 Winter, PM2.5 Monsoon, Traffic Congestion Index, num_schools, num_hospitals, num_metro_stations. Rent was intentionally excluded from clustering to prevent conflating market price with intrinsic liveability characteristics.

Cluster Validation: Optimal K confirmed by Elbow Method (inertia inflection at K=4) and Silhouette Score (global maximum S=0.41 at K=4). Four clusters labelled post-hoc as: Budget Peripheral, Mid-range Suburban, Well-connected Mid-tier, and Premium Central.

Weighted Sum Recommendation Engine: Replaces earlier Cosine Similarity implementation. Formula: Match Score = $[\text{SUM}(w_i \times f_i) / \text{SUM}(w_i)] \times 100$, where w_i is the user slider weight (0-10) and f_i is the adjusted scaled score for factor i . Output is a percentage between 0 and 100, normalised regardless of absolute weight magnitudes.

Random Forest Feature Analysis: A Random Forest regressor was trained on neighbourhood features to predict 2BHK rent, producing interpretable feature importance scores. Distance from Connaught Place (city centre) emerged as the dominant predictor (importance=0.312), consistent with urban rent gradient theory. Model R²=0.46 on 5-fold cross-validation — modest but expected given that property-level factors (floor, age, furnishing) are not captured at neighbourhood granularity.

Traffic Profiling: Composite Traffic Congestion Index built from 16 API readings per neighbourhood over 4 days, covering Morning Peak (09:00), Afternoon (13:00), Evening Peak (18:00), and Night (23:00). Manual script run at each time slot; readings appended to a cumulative traffic_log.csv; final averages computed by aggregate_traffic.py.

6. Application Development (Dashboard)

The recommendation engine is deployed as an interactive web application built with the Streamlit framework, accessible via browser at localhost.

Preference Tuner Sidebar: Six sliders labelled in plain language (Clean Air, Low Traffic, Affordable Rent, Schools Nearby, Hospitals Nearby, Metro Access), each accepting weights from 0 to 10. All 72 neighbourhoods are re-ranked in real time with every slider change.

Top-10 Ranked Table: Displays Match Score, estimated 2BHK rent, PM2.5 Winter, cluster label, and nearest metro station for the top-10 recommended neighbourhoods. Colour-gradient background highlights the score column for quick visual scanning.

Interactive Folium Map: Top-5 recommendations plotted as colour-coded circle markers on a CartoDB dark matter base map. Best match labelled with a floating name badge. Hover tooltips and click popups show neighbourhood name, score, and rent.

Per-Factor Breakdown: Horizontal bar chart for the top-matched neighbourhood showing its adjusted score (0-100) on each of the six liveability dimensions, making the reason for the recommendation transparent to the user.

Rent vs Score Bubble Chart: Scatter plot of all top-10 candidates by actual rent (x-axis) vs Match Score (y-axis), with bubble size proportional to score. Helps users see the affordability-quality trade-off at a glance.

- **Full Neighbourhood Explorer:** Searchable table of all 72 neighbourhoods with Match Score, rent, PM2.5, traffic, amenity counts, and cluster label — allowing users to look up any area directly.

7. Project Progress Table

Week	Phase	Key Activities & Progress
1–2	Architecture Design	Finalised CityMatch concept; feasibility studies for TomTom, OSMnx, and CPCB data sources; defined 72 neighbourhood centroid list.
3	Data Pipeline	Built automated OSMnx scripts for amenity density extraction (1 km radius); merged geospatial, environmental, and rental datasets into master_dataset_v1.csv.
4	Preprocessing	Applied Min-Max Scaling; implemented 95th-percentile capping for amenity outliers; reconstructed Rent_Price column from 2BHK range midpoints.
5	Traffic Integration	Integrated TomTom Flow API; designed 16-reading sampling protocol (4 days x 4 slots); computed stable Congestion Index per neighbourhood; generated master_dataset_v3.csv.
6	Rent Data Upgrade	Replaced manually estimated rents with Kaggle-sourced median 2BHK prices (5,902 listings); implemented fuzzy name matching for 24 unmatched localities; generated master_dataset_v5.csv.
7	ML Implementation	Executed K-Means Clustering (K=4, S=0.41); validated with Elbow Method and Silhouette Score; assigned post-hoc cluster labels; trained Random Forest for feature importance analysis; generated clustered_results_v5.csv.
8	Scoring Engine Rebuild	Identified and corrected Cosine Similarity directional error; rebuilt scoring engine as Weighted Sum with per-factor inversion; passed all three boundary-condition validation tests.
9	Dashboard Development	Built Streamlit UI with six-slider preference tuner, Folium map, per-factor breakdown chart, Rent vs Score bubble chart, and full neighbourhood explorer with search.
10	Research Output	Drafted IEEE-format research paper (6 pages) with 5 figures and a dashboard image placeholder; prepared Technovate conference poster; compiled full project documentation.

8. Challenges and Solutions

Challenge: OSMnx Amenity Tagging Noise

Problem: OpenStreetMap tags small dispensaries, Ayurvedic centres, and government health posts identically to full hospitals. One peripheral neighbourhood recorded 17 hospitals — the dataset maximum — causing it to dominate all recommendations regardless of user preferences.

Solution: Applied 95th-percentile capping on all amenity count features before scaling. Explicitly excluded three peri-urban settlements whose profiles were systematically incomparable to urban residential areas. Both decisions are documented as reproducible, defensible data quality steps.

Challenge: Real-Time Traffic Data Instability

Problem: A single TomTom API call captures a moment in time, not a characteristic of the neighbourhood. Traffic at 23:00 looks nothing like traffic at 09:00, making any single reading misleading as a liveability signal.

Solution: Designed a 16-reading manual sampling protocol across 4 days and 4 daily time slots. Each run appends to a cumulative traffic_log.csv; a separate aggregation script computes the stable daily average. The protocol resumes automatically if interrupted, tracking run count from the log file.

Challenge: Scoring Engine Directional Error

Problem: The original Cosine Similarity implementation produced rankings where a neighbourhood that scored perfectly on rent alone could outrank genuinely well-balanced neighbourhoods when the user maxed the rent

slider. Connaught Place (high traffic, high pollution) also scored near the top on clean-air queries — the opposite of the correct result.

Solution: Replaced Cosine Similarity with a Weighted Sum formula incorporating per-factor directional inversion ($f = 1 - f_{\text{scaled}}$ for inverse features) and weight normalisation. Three boundary-condition tests confirmed correctness before deployment.

Challenge: Rent Data Quality

Problem: Initial Rent_Price values were manually estimated midpoints of rent ranges collected by survey, introducing substantial subjectivity and compression bias. The single midpoint per neighbourhood failed to reflect actual market variance, contributing to a low Random Forest R2 of 0.458.

Solution: Replaced all estimated values with median rents aggregated from 5,902 verified 2BHK listings from a Kaggle dataset. Name matching used direct string comparison for 48 localities and manually verified fuzzy matching for the remaining 24. This also added three new columns to the dataset: Predicted_Rent, Rent_Gap, and Value_Score.

9. Pipeline File & Script Inventory

Script / File	Purpose	Output
01_osmnx_scraper.py	Extract amenity counts via OSMnx within 1 km radius	data_geospatial_amenities.csv
02_preprocessing_pipeline.py	Merge sources, fill rent via Kaggle 2BHK medians, apply 95th-pct cap + MinMaxScaler	master_dataset_v5.csv
07_fetch_traffic_manually.py	Manual TomTom API fetch (run 4x/day); appends to log	traffic_log.csv
aggregate_traffic.py	Average 16 readings into stable Traffic_Index per neighbourhood	master_dataset_v5.csv (updated)
clustering_script.py	K-Means (K=4); Elbow + Silhouette validation; cluster label assignment	clustered_results_v5.csv
train_model.py	Random Forest regressor; feature importance; Value_Score computation	citymatch_model.pkl, master_dataset_v5.csv
citymatch_app.py	Streamlit dashboard with 6-slider engine, Folium map, charts, explorer	Live web app (localhost:8501)

10. Conclusion & Forward Plan

The CityMatch intelligence layer is fully operational. The complete pipeline — from raw API data collection through preprocessing, clustering, scoring, and Streamlit deployment — has been implemented, debugged, and validated. The system correctly ranks 72 Delhi neighbourhoods in real time based on complex multi-criteria user queries.

Current Status:

- 72-neighbourhood dataset with 6 scaled liveability features — complete.
- K-Means clustering ($K=4$, $S=0.41$) with four labelled tiers — complete.
- Weighted Sum scoring engine with directional inversion — complete and validated.
- Streamlit dashboard with map, charts, and neighbourhood explorer — complete.
- Random Forest feature importance analysis — complete ($R^2=0.46$).
- IEEE research paper (6 pages) and Technovate conference poster — complete.

Next Steps:

- **Supervised Classification Layer:** Train a Random Forest classifier on cluster labels so new neighbourhoods can be automatically assigned to liveability tiers without re-running K-Means — making the system scalable to other cities.
- **NCR Expansion:** Extend coverage to Noida and Gurgaon to enable cross-city recommendations for users considering the broader Delhi NCR region.
- **Persona Testing:** Conduct structured user testing with three defined personas (Budget Professional, Young Family, Senior Couple) to validate that recommendations align with ground-truth residential knowledge.
- **Thesis Report:** Compile full technical documentation incorporating all pipeline decisions, data quality mitigations, validation results, and future work.
- **TomTom Historical API:** Replace the manual 16-reading sampling protocol with automated historical traffic statistics (where API tier permits) to eliminate the 4-day data collection constraint.

Student Signature
(Sagar Singh Bisht):

Student Signature
(Aakarsh Pab):

Faculty Guide Signature:
